



Alma Mater Studiorum-Università di Bologna
Scuola di Ingegneria

ESERCITAZIONE AUTONOMA

Display a 7 segmenti & Orologio

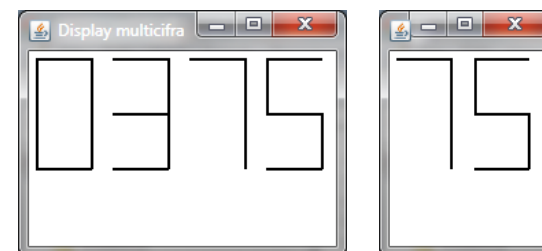
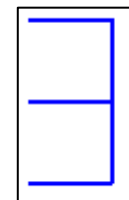
Corso di Laurea in Ingegneria Informatica
Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

IL COMPONENTE «DISPLAY A 7 SEGMENTI»

- Un componente che visualizza *una singola cifra numerica* compresa fra 0 e 9, oppure *il segno –*
- Classe **SevenSegmentDisplay**
 - metodo **setDigit** per impostare la cifra (fra 0 e 9) (un valore negativo fa comparire il segno «-»)
 - metodo **setColor** per impostare il colore
- Raramente però è sufficiente una sola cifra: sarebbe molto utile poter avere un *display multi-cifra* configurabile
- A questo provvede la *factory internalizzata* nella classe
 - metodo **of** per costruire il display
 - metodo **setValue** per impostare il valore (multi-cifra ma non negativo) da visualizzare



IL COMPONENTE «DISPLAY A 7 SEGMENTI»

Package `sevensegmentdisplay`

Class **SevenSegmentDisplay**

Constructor Summary

Constructors

Constructor	Description
<code>SevenSegmentDisplay()</code>	Costruisce un display a sette segmenti di dimensioni standard

Evitare il costruttore diretto: restituisce un display senza finestra attorno, che non sapremmo usare!

Method Summary

All Methods Static Methods Instance Methods Concrete Methods

Modifier and Type	Method	Description
<code>static SevenSegmentDisplay[]</code>	<code>of(int digits, java.lang.String title)</code>	Costruisce una finestra con un display a 7 segmenti a più cifre
<code>void</code>	<code>setColor(java.awt.Color newColor)</code>	Definisce il colore corrente usato per il disegno
<code>void</code>	<code>setDigit(int digit)</code>	Disegna la cifra richiesta, che dev'essere compresa fra 0 e 9.
<code>static void</code>	<code>setValue(int value, SevenSegmentDisplay[] displays)</code>	

Molto meglio la factory, che nasconde tutti i dettagli e ci dà una finestra con il display multi-cifra già pronto per l'uso! 😊

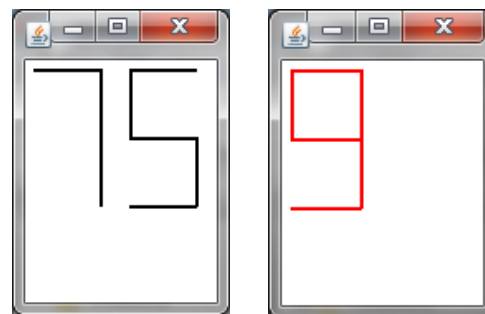
ESEMPI D'USO

```
public class Prova2 {

    public static void main(String args[]){
        SevenSegmentDisplay[] myDisplays;
        // ----- two-digit display: 75
        myDisplays = SevenSegmentDisplay.of(2, "Prova2 - 2 digits");
        myDisplays[1].setDigit(7); // decine
        myDisplays[0].setDigit(5); // unità
        //----- single digit display: 9
        myDisplays = SevenSegmentDisplay.of(1, "Prova2 - 1 digit");
        myDisplays[0].setDigit(9); // unità
        myDisplays[0].setColor(java.awt.Color.red);
    }
}
```

Crea e restituisce un display a due cifre
È un array di display (indice 0 = unità)

Crea e restituisce un
display a una cifra, rossa



```
import sevensegmentdisplay.SevenSegmentDisplay;

public class Prova3 {

    public static void main(String args[]){
        SevenSegmentDisplay[] myDisplays;
        myDisplays = SevenSegmentDisplay.of(4, "Display multicifra");
        SevenSegmentDisplay.setValue(375, myDisplays);
    }
}
```

Crea e restituisce un display a quattro cifre

Le cifre a sinistra sono impostate
automaticamente a 0



VERSO UN VERO ADT «DISPLAY»

- **SevenSegmentDisplay** è solo un punto di partenza
 - offre i metodi di base, *ma obbliga a gestire manualmente un array di display e a gestire le singole cifre una ad una*
- Serve un **vero ADT Display** che *incapsuli tutto ciò*

Package sevensegmentdisplay

Class Display

Constructor Summary

Constructors

Constructor	Description
<code>Display(int digits)</code>	Costruisce una finestra con un display a 7 segmenti a N cifre (N compreso fra 1 e 8)
<code>Display(int digits, java.lang.String title)</code>	Costruisce una finestra con un display a 7 segmenti a N cifre (N compreso fra 1 e 8), col titolo specificato

Crea e restituisce un display a N cifre (eventualmente con un titolo a nostra scelta)

Method Summary

All Methods **Instance Methods** **Concrete Methods**

Modifier and Type	Method	Description
void	<code>setColor(java.awt.Color color)</code>	Imposta il colore del display
void	<code>setValue(int value)</code>	Imposta sul display il valore specificato

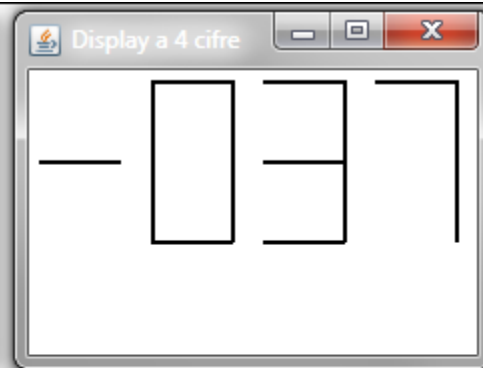
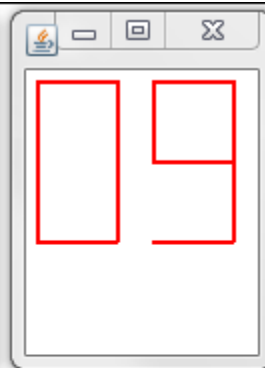
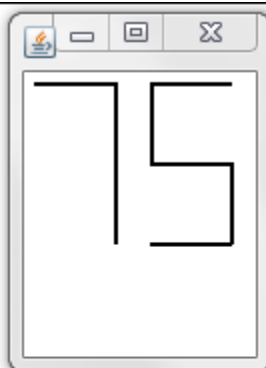
Imposta sul display il valore numerico specifico (anche negativo!)

ESEMPI D'USO

```
import sevensegmentdisplay.Display;

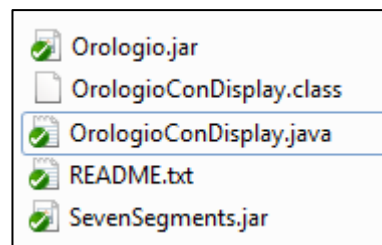
public class Prova2 {

    public static void main(String args[]){
        // ----- two-digit display: 75
        Display d = new Display(2);
        d.setValue(75);
        //----- single digit display: 9
        Display dd = new Display(2);
        dd.setColor(java.awt.Color.red);
        dd.setValue(9);
        // ----- four-digit display: 75
        Display ddd = new Display(4);
        ddd.setValue(-37);
    }
}
```



L'APPETITO VIEN MANGIANDO...

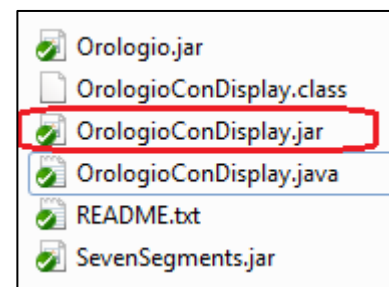
- Perché non pensare a un vero **OrologioConDisplay** ?
- È una composizione:
 - un **Orologio**
 - un **Display a 6 cifre** (ore, minuti, secondi)
PROBLEMA: non ci sono i «:» e il display mostra un valore *intero*
→ *per mostrare le 12:22:36 mostreremo in realtà l'intero 122236*
 - opportunamente *configurati e coordinati* per lavorare insieme
- Tecnicamente, servono:
 - **Orologio.jar**
 - **SevenSegments.jar**
 - e naturalmente la nuova classe!



IDEA: PERCHÉ NON FARE UN VERO JAR *ESEGUIBILE* ?

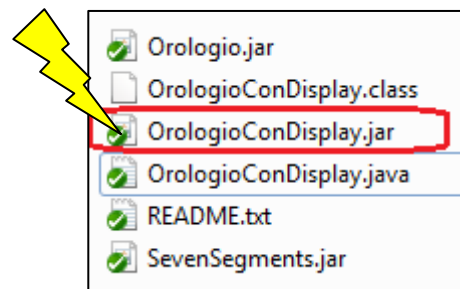
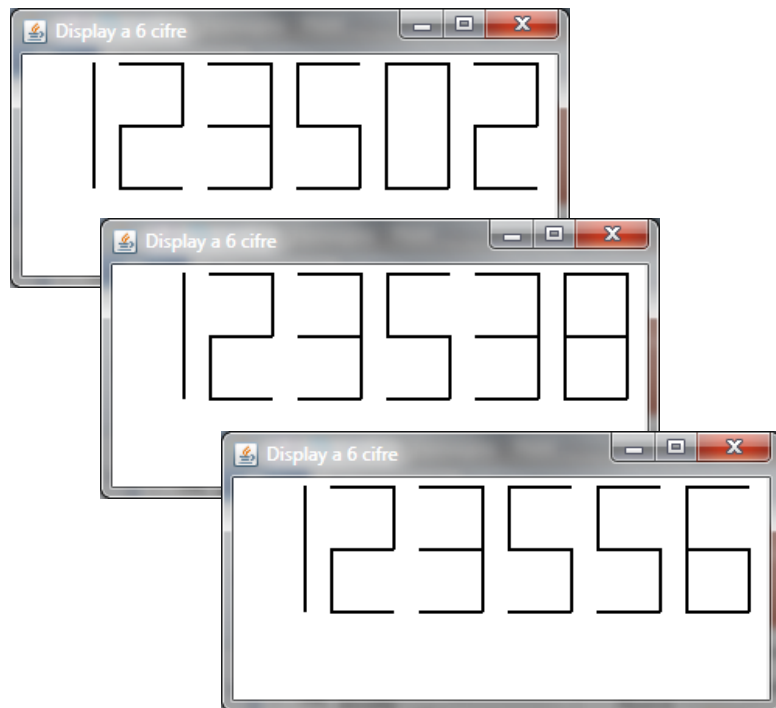
- Questa è realmente un'applicazione grafica!
 - *può davvero partire facendoci doppio clic sopra!*
- Tecnicamente, servono gli stessi elementi:
 - `Orologio.jar`
 - `SevenSegments.jar`
- MA occorre anche **creare il JAR con la nuova classe e le giuste informazioni nel MANIFEST**
 - *la classe con il main*
 - *il class path*, ossia l'elenco dei JAR necessari (separati da spazi)

```
1 Manifest-Version: 1.0
2 Created-By: 10.0.2 (Oracle Corporation)
3 Main-Class: OrologioConDisplay
4 Class-Path: Orologio.jar SevenSegments.jar
5 |
```



IDEA: PERCHÉ NON FARE UN VERO JAR *ESEGUIBILE* ?

- E infatti, se tutto è fatto bene..



- *Compromesso: non ci sono i «:» di separazione fra ore, minuti e secondi*
- D'altronde, questo è un display per valori numerici..



COME SI REALIZZA L'OROLOGIO CON DISPLAY?

- Con pochissime righe di codice!

PROVA A FARLO TU !

- *(poi se proprio non riesci o vuoi confrontare...)*

```
import sevensegmentdisplay.Display;
import java.time.LocalDateTime;
import clock.Orologio;

public class OrologioConDisplay {
    Orologio clock;
    Display display;

    public OrologioConDisplay(LocalTime t) {
        clock = new Orologio(t.getHour(), t.getMinute(), t.getSecond());
        display = new Display(6);
        display.setValue(t.getHour()*10000+t.getMinute()*100+t.getSecond());
    }

    public void tic() {
        clock.tic();
        int hh = clock.getHours(), mm = clock.getMinutes(), ss = clock.getSeconds();
        display.setValue(hh*10000+mm*100+ss);
    }

    public String toString() {
        return clock.toString();
    }

    // -----

    public static void main(String args[]){
        OrologioConDisplay orologio = new OrologioConDisplay(LocalTime.now());
        while(true){
            try {Thread.sleep(1000);} catch (InterruptedException e){}
            orologio.tic();
        }
    }
}
```